

দিন ২

Searching ও Sorting

খোঁজা আর সাজানো — প্রায় প্রতিটা ইন্টারভিউতে অন্তত একটা প্রশ্ন এখান থেকে আসে

আজ যা শিখবো

- Linear Search বনাম Binary Search
- Binary Search-এর টেমপ্লেট ও কমন ভুল
- Bubble, Selection, Counting Sort — সহজ sorting
- Merge Sort ও Quick Sort — Divide & Conquer
- Stable sort, in-place sort কী
- সব sorting-এর তুলনা টেবিল

কোর্স ভিডিও: ৮-১৩, ২৬-২৮

দিন শেষে তুমি পারবে

- Binary search না দেখে, কোনো bug ছাড়া লিখতে পারবে
- Bubble sort আর Merge sort নিজে লিখতে পারবে
- Quick sort-এর ধারণা আর worst case ব্যাখ্যা করতে পারবে
- কোন sorting কখন ভালো — তুলনা করতে পারবে

স্ট্যান্ডি শিডিউল (প্রায় ৫-৬ ঘণ্টা)

৪৫ মিনিট	Searching কনসেপ্ট + Binary Search
৪৫ মিনিট	Search Insert Position + নিজে আবার লেখা
১ ঘণ্টা	Bubble ও Selection Sort
১.৫ ঘণ্টা	Merge Sort ও Quick Sort (সবচেয়ে গুরুত্বপূর্ণ)
৩০ মিনিট	Counting Sort + তুলনা টেবিল
৪৫ মিনিট	খিওরি প্রশ্নোত্তর + চিট শিট

অধ্যায় ১: Searching — খোঁজার দুই উপায়

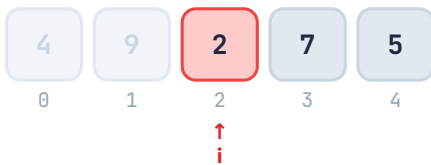
Linear Search

একদম শুরু থেকে একটা একটা করে দেখো, যতক্ষণ না পাও। Array sorted হোক বা না হোক — কাজ করে।

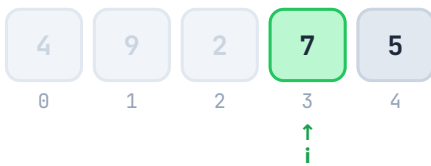
ধাপ 1 7 খুঁজছি। index 0: 4 ≠ 7



ধাপ 2 index 1: 9 ≠ 7, index 2: 2 ≠ 7



ধাপ 3 index 3: 7 == 7 → পেয়েছি! রিটার্ন 3



PHP

```
function linearSearch(array $arr, int $target): int {
    foreach ($arr as $i => $value) {
        if ($value === $target) return $i;
    }
    return -1; // পাওয়া যায়নি
}
echo linearSearch([4, 9, 2, 7, 5], 7); // 3
```

Time: $O(n)$

Space: $O(1)$

Binary Search

শর্ত: array অবশ্যই sorted হতে হবে। মাঝখানের মান দেখো। টার্গেট তার চেয়ে বড় হলে বাম অর্ধেক পুরোটা বাদ, ছোট হলে ডান অর্ধেক বাদ। প্রতি ধাপে অর্ধেক বাদ যায় — তাই $O(\log n)$ ।

বাস্তব উদাহরণ: অভিধানে শব্দ খোঁজা

"মাছ" শব্দটা খুঁজতে তুমি অভিধানের প্রথম পাতা থেকে পড়া শুরু করো না। মাঝখান খোলো — "ত" পেলে বুঝবে "ম" পরে আছে, সামনের অর্ধেক বাদ। আবার বাকি অংশের মাঝখান... কয়েকবারেই পৌঁছে যাও।

Array সাইজ	Linear (worst)	Binary (worst)
100	100 বার	7 বার
10,000	10,000 বার	14 বার
10,00,000	10 লাখ বার	20 বার
100 কোটি	100 কোটি বার	30 বার

Binary Search-এর ৩টা কমন bug

1. **লুপের শর্ত:** `while ($low <= $high)` — `<=` দিতে হবে, নইলে শেষ একটা element চেক হয় না।
2. **অসীম লুপ:** `$low = $mid + 1` আর `$high = $mid - 1` — শুধু `$mid` দিলে আটকে যেতে পারে।
3. **Overflow:** অনেক ভাষায় `(low + high) / 2` overflow হয়। নিরাপদ: `low + intval(high - low, 2) / 2`।
PHP-তে বড় int float হয়ে যায়, তাই এটা বললেই ইন্টারভিউয়ার খুশি।

অধ্যায় ২: Sorting — কেন আর কোনটা

Sorting মানে ছোট থেকে বড় (বা উল্টো) ক্রমে সাজানো। Sorted data-তে binary search, duplicate খোঁজা, two pointer — সব সহজ হয়ে যায়।

দুইটা গুরুত্বপূর্ণ শব্দ

শব্দ	মানে	উদাহরণ
In-place	আলাদা বড় array ছাড়াই ($O(1)$ বা $O(\log n)$ extra space) সাজায়	Bubble, Selection, Quick sort
Stable	সমান মানের element-দের আগের ক্রম ঠিক থাকে	Bubble, Merge sort — stable; Selection, Quick — না

Stable কেন দরকার?

ধরো ছাত্রদের আগে নাম অনুযায়ী সাজানো আছে। এখন নম্বর অনুযায়ী সাজালে, সমান নম্বর পাওয়া ছাত্ররা যেন নাম-ক্রমেই থাকে — এটা stable sort গ্যারান্টি দেয়। ই-কমার্চে "দাম অনুযায়ী, সমান দামে রেটিং অনুযায়ী" সাজাতে এটা লাগে।

Sorting বড় ছবি

অ্যালগরিদম	মূল ধারণা	গড় Time	কখন
Bubble	পাশাপাশি দুইটা তুলনা, ভুল ক্রমে থাকলে swap	$O(n^2)$	শেখার জন্য, প্রায় sorted ডেটা
Selection	বাকি অংশ থেকে সবচেয়ে ছোটটা বেছে সামনে আনো	$O(n^2)$	swap কম করতে হলে
Merge	অর্ধেক করো, আলাদা সাজাও, জোড়া লাগাও	$O(n \log n)$	stable আর গ্যারান্টি দরকার
Quick	pivot বেছে ছোটদের বামে, বড়দের ডানে	$O(n \log n)$	বাস্তবে সবচেয়ে দ্রুত, in-place
Counting	প্রতিটা মান কতবার আছে গুনো	$O(n + k)$	ছোট রেঞ্জের integer (যেমন বয়স, নম্বর)

প্রশ্ন 2.1

Binary Search

Sorted array-তে একটা সংখ্যার index খোঁজো

Easy

LeetCode 704

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

সবচেয়ে গুরুত্বপূর্ণ সার্চিং অ্যালগরিদম। ইন্টারভিউতে প্রায়ই বলে "bug-free binary search লেখো"। অনেক Medium প্রশ্নের ভিত্তি।

সমস্যাটা কী?

একটা **ascending sorted** integer array `nums` আর `target` দেওয়া আছে। `target` থাকলে তার index রিটার্ন করো, না থাকলে -1। $O(\log n)$ -এ করতে হবে।

উদাহরণ

Input: `nums = [-1, 0, 3, 5, 9, 12]`, `target = 9`

Output: 4

Input: `nums = [-1, 0, 3, 5, 9, 12]`, `target = 2`

Output: -1

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

"আমি ১ থেকে ১০০-র মধ্যে একটা সংখ্যা ভেবেছি" খেলা। বুদ্ধিমান লোক প্রথমে বলে ৫০। "বড়" শুনলে বলে ৭৫। প্রতিবার অর্ধেক বাদ — ৭ বারের মধ্যেই সংখ্যা ধরে ফেলা যায়।

পদ্ধতি ১: Linear Search

শুরু থেকে একটা একটা করে দেখা। কাজ করে, কিন্তু sorted হওয়ার সুবিধা ব্যবহার করে না।

PHP

```
function searchLinear(array $nums, int $target): int {
    for ($i = 0; $i < count($nums); $i++) {
        if ($nums[$i] === $target) return $i;
    }
    return -1;
}
echo searchLinear([-1, 0, 3, 5, 9, 12], 9); // 4
```

Time: $O(n)$ Space: $O(1)$

পদ্ধতি ২: Binary Search (Iterative) — এটাই মুখস্থ রাখবে

1. `low = 0`, `high = n - 1`
2. যতক্ষণ `low ≤ high`: `mid = low + (high - low) / 2`

3. `nums[mid] == target` → mid রিটার্ন
4. `nums[mid] < target` → ডানে যাও: `low = mid + 1`
5. নইলে বামে: `high = mid - 1`

PHP

```
function binarySearch(array $nums, int $target): int {
    $low = 0;
    $high = count($nums) - 1;
    while ($low <= $high) {
        $mid = $low + intval($high - $low, 2); // <= খেয়াল করো
        if ($nums[$mid] === $target) { // overflow-নিরাপদ
            return $mid; // পেয়েছি
        } elseif ($nums[$mid] < $target) {
            $low = $mid + 1; // বাম অর্ধেক বাদ
        } else {
            $high = $mid - 1; // ডান অর্ধেক বাদ
        }
    }
    return -1;
}
echo binarySearch([-1, 0, 3, 5, 9, 12], 9); // 4
echo "\n" . binarySearch([-1, 0, 3, 5, 9, 12], 2); // -1
```

Time: $O(\log n)$ Space: $O(1)$

পদ্ধতি ৩: Binary Search (Recursive) — ইন্টারভিউতে চাইতে পারে

একই যুক্তি, কিন্তু লুপের বদলে ফাংশন নিজেকে ছোট রেঞ্জ দিয়ে কল করে। Base case: `low > high` মানে পাওয়া যায়নি।

PHP

```
function binarySearchRec(array $nums, int $target, int $low, int $high): int {
    if ($low > $high) return -1; // base case
    $mid = $low + intval($high - $low, 2);
    if ($nums[$mid] === $target) return $mid;
    if ($nums[$mid] < $target) {
        return binarySearchRec($nums, $target, $mid + 1, $high);
    }
    return binarySearchRec($nums, $target, $low, $mid - 1);
}
$a = [-1, 0, 3, 5, 9, 12];
echo binarySearchRec($a, 5, 0, count($a) - 1); // 3
```

Time: $O(\log n)$ Space: $O(\log n)$

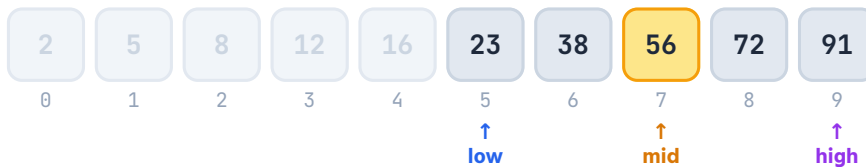
Recursion stack-এ $\log n$ টা কল জমে, তাই space $O(\log n)$ । Iterative-টা $O(1)$ — তাই সাধারণত iterative ভালো।

ডাইরান — ধাপে ধাপে দেখি

ধাপ 1 arr = [2,5,8,12,16,23,38,56,72,91], target = 23। low=0, high=9 → mid=4, arr[4]=16 < 23 → বাম অর্ধেক বাদ, low=5



ধাপ 2 low=5, high=9 → mid=7, arr[7]=56 > 23 → ডান অর্ধেক বাদ, high=6



ধাপ 3 low=5, high=6 → mid=5, arr[5]=23 == 23 → পেয়েছি! মাত্র ৩ ধাপে (linear-এ ৬ ধাপ লাগতো)



এজ কেস (এগুলো ভুলবে না)

- খালি array → high = -1, লুপ চলে না, -1 রিটার্ন
- এক element
- target সবচেয়ে ছোট বা সবচেয়ে বড়ের বাইরে
- ডুপ্লিকেট থাকলে যেকোনো একটা index আসবে

ইন্টারভিউতে যা বলবে

- শুরুতেই জিজ্ঞেস করো: "array কি sorted?" — না হলে binary search চলবে না।
- বলো কেন $O(\log n)$: "প্রতি ধাপে সার্চ স্পেস অর্ধেক হয়, $n \rightarrow n/2 \rightarrow n/4 \dots \rightarrow 1$, তাই $\log_2 n$ ধাপ।"

ফলো-আপ প্রশ্ন (ইন্টারভিউয়ার জিজ্ঞেস করতে পারে)

"ডুপ্লিকেট থাকলে প্রথম occurrence খোঁজো" — পেলেও থেমনো না: `$ans = $mid; $high = $mid - 1;` দিয়ে বামে খুঁজতে থাকো। (LeetCode 34)

প্রশ্ন 2.2

Search Insert Position

Target না থাকলে কোথায় বসবে তার index

Easy

LeetCode 35

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

Binary search-এর সবচেয়ে কমন variation। শেখায় লুপ শেষে low ঠিক কোথায় থাকে — এই জ্ঞান অনেক প্রশ্নে লাগে।

সমস্যাটা কী?

Sorted array আর target দেওয়া আছে। target থাকলে তার index, না থাকলে sorted ক্রম ঠিক রেখে যেখানে ঢোকানো উচিত সেই index রিটার্ন করো।

উদাহরণ

Input: nums = [1, 3, 5, 6], target = 5

Output: 2

Input: nums = [1, 3, 5, 6], target = 2

Output: 1

Input: nums = [1, 3, 5, 6], target = 7

Output: 4

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

লাইব্রেরির সাজানো তাকে নতুন একটা বই রাখবে। কোথায় রাখবে? ঠিক সেই জায়গায় যেখানে আগের বই ছোট আর পরের বই বড়।

পদ্ধতি ১: Linear — প্রথম যে মান $\geq$ target

শুরু থেকে হাঁটো। প্রথম যে element target-এর সমান বা বড়, তার index-ই উত্তর। কেউ না হলে n।

PHP

```
function searchInsertLinear(array $nums, int $target): int {
    foreach ($nums as $i => $x) {
        if ($x >= $target) return $i;
    }
    return count($nums);
}
echo searchInsertLinear([1, 3, 5, 6], 2); // 1
```

Time: $O(n)$ Space: $O(1)$

পদ্ধতি ২: Binary Search — লুপ শেষে low-ই উত্তর

1. সাধারণ binary search চালাও
2. পেলে mid রিটার্ন
3. না পেলে লুপ শেষে low রিটার্ন — এটাই insert position

PHP

```
function searchInsert(array $nums, int $target): int {
    $low = 0;
    $high = count($nums) - 1;
    while ($low <= $high) {
        $mid = $low + intval($high - $low, 2);
        if ($nums[$mid] === $target) return $mid;
        if ($nums[$mid] < $target) {
            $low = $mid + 1;
        } else {
            $high = $mid - 1;
        }
    }
    return $low; // low = প্রথম index যেখানে মান > target
}
```

```
echo searchInsert([1, 3, 5, 6], 5); // 2
echo " " . searchInsert([1, 3, 5, 6], 2); // 1
echo " " . searchInsert([1, 3, 5, 6], 7); // 4
```

Time: $O(\log n)$ Space: $O(1)$

ড্রাই রান — ধাপে ধাপে দেখি

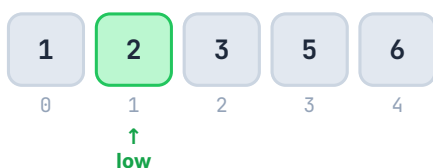
ধাপ 1 $nums=[1,3,5,6]$, $target=2$ । $low=0$, $high=3$, $mid=1$: $3 > 2 \rightarrow high=0$



ধাপ 2 $low=0$, $high=0$, $mid=0$: $1 < 2 \rightarrow low=1$



ধাপ 3 $low=1 > high=0 \rightarrow$ লুপ শেষ। $low=1$ রিটার্ন — 2 বসবে 1 আর 3-এর মাঝে



এজ কেস (এগুলো ভুলবে না)

- target সবার ছোট $\rightarrow 0$
- target সবার বড় $\rightarrow n$
- খালি array $\rightarrow 0$

ইন্টারভিউতে যা বলবে

- বলো: "লুপ শেষ হলে low সবসময় প্রথম সেই পজিশনে থাকে যেখানে মান target-এর চেয়ে বড় — এটাকে lower bound বলে।"

প্রশ্ন 2.3

Bubble Sort

পাশাপাশি তুলনা করে swap — সবচেয়ে সহজ sorting

Easy

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

ইন্টারভিউতে "একটা sorting অ্যালগরিদম লেখো (sort() ছাড়া)" বললে প্রায় সবাই এটা লেখে। Optimized ভার্সন (early exit) জানলে আলাদা নম্বর পাবে।

সমস্যাটা কী?

একটা integer array **ascending** ক্রমে সাজাও, কোনো built-in sort ফাংশন ছাড়া।

উদাহরণ

Input: [5, 1, 4, 2, 8]

Output: [1, 2, 4, 5, 8]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

পানিতে বুদবুদ যেমন উপরে উঠে আসে — প্রতিটা পাসে সবচেয়ে বড় সংখ্যাটা "ভেসে" array-র শেষে চলে যায়। প্রথম পাসে সবচেয়ে বড়, দ্বিতীয় পাসে দ্বিতীয় বড়...

পদ্ধতি ১: সাধারণ Bubble Sort

- বাইরের লুপ: $n-1$ টা পাস
- ভেতরের লুপ: পাশাপাশি দুইটা ($j, j+1$) তুলনা, বামেরটা বড় হলে swap
- প্রতিটা পাস শেষে শেষের i টা ঘর ঠিক জায়গায় — তাই ভেতরের লুপ $n - 1 - i$ পর্যন্ত

PHP

```
function bubbleSort(array $arr): array {
    $n = count($arr);
    for ($i = 0; $i < $n - 1; $i++) {
        for ($j = 0; $j < $n - 1 - $i; $j++) {           // শেষের i টা ইতিমধ্যে ঠিক
            if ($arr[$j] > $arr[$j + 1]) {
                [$arr[$j], $arr[$j + 1]] = [$arr[$j + 1], $arr[$j]];
            }
        }
    }
    return $arr;
}
echo implode(", ", bubbleSort([5, 1, 4, 2, 8])); // 1, 2, 4, 5, 8
```

Time: $O(n^2)$ সবসময়Space: $O(1)$

পদ্ধতি ২: Optimized — কোনো swap না হলে থামো

- প্রতিটা পাসে একটা `$swapped` flag রাখো
- পুরো পাসে একটাও swap না হলে array ইতিমধ্যে sorted → break

PHP

```
function bubbleSortOptimized(array $arr): array {
    $n = count($arr);
    for ($i = 0; $i < $n - 1; $i++) {
        $swapped = false;
        for ($j = 0; $j < $n - 1 - $i; $j++) {
            if ($arr[$j] > $arr[$j + 1]) {
                [$arr[$j], $arr[$j + 1]] = [$arr[$j + 1], $arr[$j]];
                $swapped = true;
            }
        }
        if (!$swapped) break; // sorted হয়ে গেছে, আর ঘোরার দরকার নেই
    }
    return $arr;
}

echo implode(" ", bubbleSortOptimized([5, 1, 4, 2, 8])); // 1, 2, 4, 5, 8
echo "\n" . implode(" ", bubbleSortOptimized([1, 2, 3])); // 1 পাসেই শেষ
```

Time: Worst $O(n^2)$, Best $O(n)$ Space: $O(1)$ আগে থেকেই sorted array-তে প্রথম পাসেই থেমে যায় → best case $O(n)$!

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 পাস ১: (5,1) swap → (5,4) swap → (5,2) swap → (5,8) ঠিক আছে

পাস ১ শেষে: সবচেয়ে বড় 8 শেষে



ধাপ 2 পাস ২: (1,4) ঠিক → (4,2) swap → (4,5) ঠিক

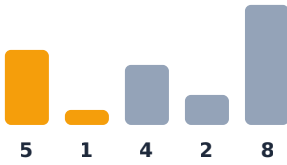
পাস ২ শেষে: 5 আর 8 ঠিক জায়গায়



ধাপ 3 পাস ৩: কোনো swap হলো না → optimized ভার্সন এখানেই থামে!



শুরুতে: প্রথম জোড়া (5, 1) তুলনা হচ্ছে



এজ কেস (এগুলো ভুলবে না)

- খালি বা এক element → লুপ চলে না
- ইতিমধ্যে sorted → optimized ভার্সনে $O(n)$
- উল্টো sorted [5,4,3,2,1] → worst case, সবচেয়ে বেশি swap

ইন্টারভিউতে যা বলবে

- বলো Bubble sort stable আর in-place, কিন্তু $O(n^2)$ বলে বড় ডেটায় ব্যবহার হয় না।

প্রশ্ন 2.4

Selection Sort

প্রতিবার সবচেয়ে ছোটটা বেছে সামনে আনো

Easy

কেন এটা গুরুত্বপূর্ণ?

Bubble sort-এর সাথে তুলনা করতে জিজ্ঞেস করে। swap সংখ্যা কম (সর্বোচ্চ $n-1$) — এটা এর বিশেষত্ব।

সমস্যাটা কী?

Selection sort দিয়ে array ascending সাজাও।

উদাহরণ

Input: [64, 25, 12, 22, 11]

Output: [11, 12, 22, 25, 64]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

তাস হাতে নিয়ে সাজাচ্ছো: পুরো হাত দেখে সবচেয়ে ছোট তাসটা বের করে বামে রাখো। তারপর বাকিদের মধ্যে সবচেয়ে ছোট... এভাবে।

Selection Sort

1. $i = 0$ থেকে $n-2$ পর্যন্ত
2. i থেকে শেষ পর্যন্ত সবচেয়ে ছোটটার index (minIdx) খোঁজো
3. $arr[i]$ আর $arr[minIdx]$ swap

PHP

```
function selectionSort(array $arr): array {
    $n = count($arr);
    for ($i = 0; $i < $n - 1; $i++) {
        $minIdx = $i;
        for ($j = $i + 1; $j < $n; $j++) {
            if ($arr[$j] < $arr[$minIdx]) $minIdx = $j;
        }
        if ($minIdx !== $i) {
            [$arr[$i], $arr[$minIdx]] = [$arr[$minIdx], $arr[$i]];
        }
    }
    return $arr;
}

echo implode(", ", selectionSort([64, 25, 12, 22, 11])); // 11, 12, 22, 25, 64
```

Time: $O(n^2)$ সবসময় (best case-ও)Space: $O(1)$

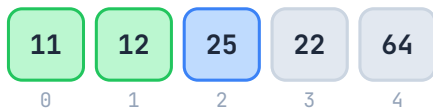
Stable না: [4a, 4b, 1] → 1 আর 4a swap হলে 4a, 4b-এর পরে চলে যায়।

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 $i=0$: সবচেয়ে ছোট 11 (index 4) → swap(0,4)



ধাপ 2 $i=1$: বাকিদের মধ্যে ছোট 12 (index 2) → swap(1,2)



ধাপ 3 $i=2$: ছোট 22 (index 3) → swap(2,3)



ধাপ 4 $i=3$: ছোট 25 নিজেই → swap লাগে না। শেষ!



ইন্টারভিউতে যা বলবে

- Bubble বনাম Selection: দুটোই $O(n^2)$ । Bubble stable ও sorted ভেটায় $O(n)$ হতে পারে; Selection-এ swap কম, কিন্তু সবসময় $O(n^2)$ আর stable না।

প্রশ্ন 2.5

Merge Sort

ভাগ করো, সাজাও, জোড়া লাগাও (Divide & Conquer)

Medium

LeetCode 912

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

$O(n \log n)$ গ্যারান্টি দেওয়া stable sort। ইন্টারভিউতে "merge sort লেখো এবং complexity ব্যাখ্যা করো" খুব কমন। Recursion বোঝার সেরা উদাহরণ।

সমস্যাটা কী?

Merge sort ব্যবহার করে array ascending ক্রমে সাজাও।

উদাহরণ

Input: [8, 3, 5, 1, 9, 2, 7, 4]

Output: [1, 2, 3, 4, 5, 7, 8, 9]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

৮০০ খাতা একা সাজানো কঠিন। অর্ধেক করে দুই শিক্ষককে দাও, তারা আবার অর্ধেক করে অন্যদের দেয়... শেষে প্রত্যেকের হাতে ১টা খাতা (নিজে নিজেই sorted!)। এখন দুইটা sorted গাদা একসাথে মেলানো খুব সহজ — দুই গাদার উপরের খাতা তুলনা করে ছোটটা নাও।

Merge Sort (Recursive)

1. **Base case:** array-তে 0 বা 1 element থাকলে সেটা ইতিমধ্যে sorted
2. **Divide:** মাঝখান থেকে দুই ভাগ করো
3. **Conquer:** দুই ভাগকে recursively sort করো
4. **Combine:** দুইটা sorted অংশকে merge করো (two pointer দিয়ে)

PHP

```
function mergeSort(array $arr): array {
    $n = count($arr);
    if ($n <= 1) return $arr; // base case

    $mid = intval($n, 2);
    $left = mergeSort(array_slice($arr, 0, $mid)); // বাম অর্ধেক sort
    $right = mergeSort(array_slice($arr, $mid)); // ডান অর্ধেক sort
    return merge($left, $right);
}

function merge(array $left, array $right): array {
    $result = [];
    $i = 0; $j = 0;
    while ($i < count($left) && $j < count($right)) {
        if ($left[$i] <= $right[$j]) { // <= রাখলে stable থাকে
            $result[] = $left[$i++];
        } else {
            $result[] = $right[$j++];
        }
    }
    while ($i < count($left)) $result[] = $left[$i++]; // বাকিগুলো
    while ($j < count($right)) $result[] = $right[$j++];
    return $result;
}

echo implode(", ", mergeSort([8, 3, 5, 1, 9, 2, 7, 4])); // 1, 2, 3, 4, 5, 7, 8, 9
```

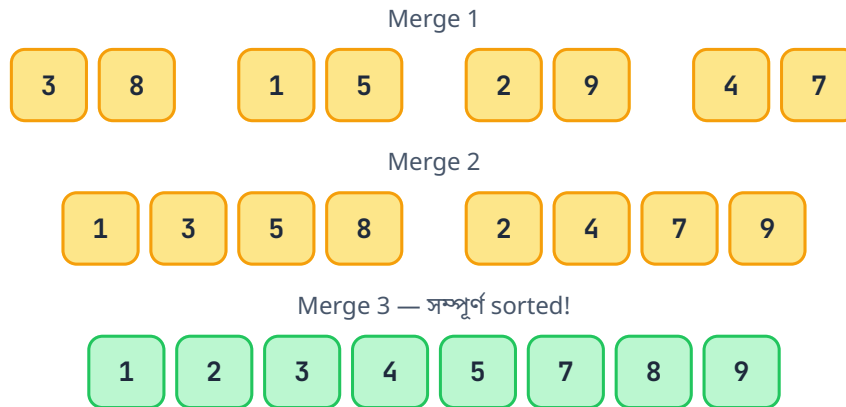
Time: $O(n \log n)$ — best, average, worst সবসময়Space: $O(n)$ কেন $n \log n$? array $\log n$ বার অর্ধেক হয় (লেভেল সংখ্যা), আর প্রতিটা লেভেলে merge করতে মোট n কাজ $\rightarrow n \times \log n$

ড্রাই রান — ধাপে ধাপে দেখি

ভাগ করার ধাপ (Divide):



জোড়া লাগানোর ধাপ (Merge):



merge() ফাংশন ভেতরে কীভাবে কাজ করে (শেষ ধাপ):

ধাপ 1 left[i]=1 বনাম right[j]=2 → 1 ছোট, result-এ নাও, i++



ধাপ 2 3 বনাম 2 → 2 নাও, j++



ধাপ 3 3 বনাম 4 → 3 নাও... এভাবে চলতে থাকে। যেকোনো একটা শেষ হলে অন্যটার বাকি সব কপি করো।



এজ কেস (এগুলো ভুলবে না)

- খালি বা এক element → সরাসরি রিটার্ন
- ডুপ্লিকেট → `<=` ব্যবহার করায় stable
- ইতিমধ্যে sorted → তবুও $O(n \log n)$

ইন্টারভিউতে যা বলবে

- Merge sort-এর দুর্বলতা বলো: $O(n)$ extra memory লাগে। Linked list sort করতে এটা সেরা (extra space ছাড়াই merge করা যায়)।
- সুবিধা: গ্যারান্টেড $O(n \log n)$ আর stable।

প্রশ্ন 2.6

Quick Sort

Pivot বেছে ভাগ করো — বাস্তবে সবচেয়ে দ্রুত sort

Medium

★ Must-do

কেন এটা গুরুত্বপূর্ণ?

অনেক ভাষার built-in sort এর উপর ভিত্তি করে তৈরি। ইন্টারভিউতে প্রশ্ন আসে: "Quick sort-এর worst case কখন আর কীভাবে এড়াবে?"

সমস্যাটা কী?

Quick sort দিয়ে array ascending সাজাও (in-place)।

উদাহরণ

Input: [7, 2, 1, 6, 8, 5, 3, 4]

Output: [1, 2, 3, 4, 5, 6, 7, 8]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

ক্লাসে একজন ছাত্রকে (pivot) দাঁড় করাও। বলো: "যারা তার চেয়ে খাটো, বামে যাও; যারা লম্বা, ডানে।" এখন pivot ঠিক তার আসল জায়গায় আছে! এবার বাম দল আর ডান দলে একই কাজ আলাদাভাবে করো।

Quick Sort (Lomuto Partition)

1. শেষ element-কে **pivot** ধরো
2. **Partition:** pivot-এর চেয়ে ছোট সবাইকে বামে আনো; শেষে pivot-কে তাদের ঠিক পরে বসাও
3. এখন pivot তার চূড়ান্ত জায়গায়
4. pivot-এর বাম আর ডান অংশে recursively একই কাজ

PHP

```

function quickSort(array &$arr, int $low, int $high): void {
    if ($low < $high) {
        $p = partition($arr, $low, $high); // pivot এখন index p-তে ঠিক জায়গায়
        quickSort($arr, $low, $p - 1);    // বাম অংশ
        quickSort($arr, $p + 1, $high);    // ডান অংশ
    }
}

function partition(array &$arr, int $low, int $high): int {
    $pivot = $arr[$high];
    $store = $low; // পরের "ছোট" element কোথায় বসবে
    for ($j = $low; $j < $high; $j++) {
        if ($arr[$j] < $pivot) {
            [$arr[$store], $arr[$j]] = [$arr[$j], $arr[$store]];
            $store++;
        }
    }
    [$arr[$store], $arr[$high]] = [$arr[$high], $arr[$store]]; // pivot বসাত
    return $store;
}

$a = [7, 2, 1, 6, 8, 5, 3, 4];
quickSort($a, 0, count($a) - 1);
echo implode(" ", $a); // 1, 2, 3, 4, 5, 6, 7, 8

```

Time: Average $O(n \log n)$, Worst $O(n^2)$ Space: $O(\log n)$ (recursion stack)

Worst case: pivot সবসময় সবচেয়ে ছোট/বড় হলে (যেমন আগে থেকেই sorted array আর শেষ element pivot) — প্রতিবার মাত্র ১টা করে কমে $\rightarrow O(n^2)$ । সমাধান: random pivot বা "median of three"।

ড্রাই রান — ধাপে ধাপে দেখি

ধাপ 1 pivot = 4 (শেষ element)। store=0। j=0: $7 \geq 4$, কিছু না



ধাপ 2 j=1: $2 < 4 \rightarrow \text{swap}(\text{store}=0, 1)$, store=1। j=2: $1 < 4 \rightarrow \text{swap}(1, 2)$, store=2



ধাপ 3 j=3,4,5: 6, 8, 5 সবাই $\geq 4 \rightarrow$ কিছু না। j=6: $3 < 4 \rightarrow \text{swap}(2, 6)$, store=3



ধাপ 4 লুপ শেষ। pivot-কে store(3)-এ বসাও: swap(3, 7)। এখন 4-এর বামে সব ছোট, ডানে সব বড়!



ধাপ 5 এবার [2,1,3] আর [8,5,7,6] আলাদাভাবে একই পদ্ধতিতে sort হবে → [1,2,3,4,5,6,7,8]



এজ কেস (এগুলো ভুলবে না)

- ইতিমধ্যে sorted array → শেষ element pivot হলে worst case $O(n^2)$
- অনেক ডুপ্লিকেট → এই ভাঙ্গন ধীর হতে পারে (3-way partition দিয়ে সমাধান)

ইন্টারভিউতে যা বলবে

- Merge বনাম Quick: Quick in-place ($O(\log n)$ space) আর বাস্তবে cache-friendly তাই দ্রুত; Merge stable আর worst case-এও $O(n \log n)$ ।
- বলো: "Random pivot নিলে worst case হওয়ার সম্ভাবনা প্রায় শূন্য।"

প্রশ্ন 2.7

Counting Sort

গুনে গুনে সাজাও — তুলনা ছাড়াই

Easy

কেন এটা গুরুত্বপূর্ণ?

তুলনা ছাড়া sorting-এর উদাহরণ। "ছোট রেঞ্জের সংখ্যা $O(n)$ -এ সাজাও" ধরনের প্রশ্নে এই ধারণা লাগে (যেমন Sort Colors)।

সমস্যাটা কী?

Non-negative integer-এর array (সর্বোচ্চ মান k) counting sort দিয়ে সাজাও।

উদাহরণ

Input: [4, 2, 2, 8, 3, 3, 1]

Output: [1, 2, 2, 3, 3, 4, 8]

সহজ ভাষায় বুঝি (বাস্তব উদাহরণ)

পরীক্ষায় ১০০ ছাত্রের নম্বর ০-১০ এর মধ্যে। তুলনা না করে ১১টা বাক্স বানাও (০ থেকে ১০)। প্রতিটা খাতা তার নম্বরের বাক্সে ফেলো। শেষে বাক্স ০ থেকে ১০ ক্রমে খালি করো — sorted!

Counting Sort

1. সর্বোচ্চ মান k বের করো, $k+1$ সাইজের count array বানাও (সব 0)
2. প্রতিটা সংখ্যার count বাড়ানো
3. count array 0 থেকে k পর্যন্ত ঘুরে, প্রতিটা মান তার count বার output-এ লেখো

PHP

```
function countingSort(array $arr): array {
    if (empty($arr)) return [];
    $max = max($arr);
    $count = array_fill(0, $max + 1, 0);
    foreach ($arr as $x) {
        $count[$x]++; // প্রতিটা মান কতবার
    }
    $result = [];
    for ($v = 0; $v <= $max; $v++) {
        for ($c = 0; $c < $count[$v]; $c++) {
            $result[] = $v;
        }
    }
    return $result;
}
echo implode(" ", countingSort([4, 2, 2, 8, 3, 3, 1])); // 1, 2, 2, 3, 3, 4, 8
```

Time: $O(n + k)$ Space: $O(k)$

k (সর্বোচ্চ মান) বিশাল হলে (যেমন 10^9) এটা অকার্যকর — তখন সাধারণ sort ভালো।

ড্রাই রান — ধাপে ধাপে দেখি

ইনপুট



count array: index = মান, ঘরের সংখ্যা = কতবার এসেছে



আউটপুট: count অনুযায়ী ক্রমে লিখলাম



ইন্টারভিউতে যা বলবে

- বলো কখন ব্যবহার করবে: integer, ছোট রেঞ্জ (বয়স ০-১২০, নম্বর ০-১০০)। নেগেটিভ থাকলে offset (min বিয়োগ) করতে হয়।

থিওরি প্রশ্নোত্তর (ইন্টারভিউ স্টাইল)

প্রতিটা উত্তর ইন্টারভিউতে ৩০-৬০ সেকেন্ডে বলার মতো করে লেখা। মুখস্থ না করে নিজের ভাষায় বলার প্র্যাকটিস করো।

প্রশ্ন 1: Linear Search আর Binary Search-এর পার্থক্য কী?

	Linear	Binary
শর্ত	কিছু না	Array sorted হতে হবে
Time	$O(n)$	$O(\log n)$
কীভাবে	একে একে দেখা	প্রতিবার অর্ধেক বাদ
কখন	ছোট বা unsorted ডেটা	বড় sorted ডেটা

প্রশ্ন 2: Binary search কেন $O(\log n)$?

প্রতি ধাপে সার্চের জায়গা অর্ধেক হয়: $n, n/2, n/4 \dots 1$ । n কে কতবার ২ দিয়ে ভাগ করলে ১ হয় — সেটাই $\log_2 n$ । ১০ লাখ element-এ মাত্র ~ 20 ধাপ।

প্রশ্ন 3: Unsorted array-তে binary search করতে চাইলে কী করবে?

আগে sort করতে হবে — $O(n \log n)$ । একবার খুঁজলে linear search ($O(n)$) ভালো। কিন্তু অনেকবার খুঁজতে হলে একবার sort করে প্রতিবার binary search লাভজনক।

প্রশ্ন 4: Stable sort কী? উদাহরণ দাও।

সমান মানের element-দের আগের আপেক্ষিক ক্রম ঠিক থাকলে stable। Merge sort, Bubble sort stable; Quick sort, Selection sort সাধারণত stable না। দরকার হয় multi-key sorting-এ — যেমন আগে নামে, তারপর বিভাগে সাজানো।

প্রশ্ন 5: Merge Sort আর Quick Sort-এর তুলনা করো।

	Merge Sort	Quick Sort
Average	$O(n \log n)$	$O(n \log n)$
Worst	$O(n \log n)$	$O(n^2)$
Space	$O(n)$	$O(\log n)$
Stable	হ্যাঁ	না
বাস্তবে	Linked list, external sort	Array-তে সাধারণত দ্রুততর

প্রশ্ন 6: Quick sort-এর worst case কখন হয় আর কীভাবে এড়াবে?

যখন pivot প্রতিবার সবচেয়ে ছোট বা বড় হয় — যেমন sorted array-তে শেষ element pivot। তখন partition প্রতিবার মাত্র ১টা element কমায়, মোট $O(n^2)$ । এড়াতে random pivot বা median-of-three (প্রথম, মাঝ, শেষের মধ্যমা) ব্যবহার করি।

প্রশ্ন 7: Merge sort কেন $O(n \log n)$?

Array ভাগ হতে হতে $\log n$ টা লেভেল হয়। প্রতিটা লেভেলে সব অংশ মিলিয়ে merge করতে মোট n কাজ লাগে। তাই $n \times \log n$ ।

প্রশ্ন 8: PHP-এর sort() কোন অ্যালগরিদম ব্যবহার করে?

PHP ভেতরে একটা hybrid sort ব্যবহার করে (ছোট অংশে insertion sort, বড় অংশে quick sort ভিত্তিক), গড়ে $O(n \log n)$ । PHP 8.0 থেকে sorting stable। ইন্টারভিউতে বলবে: "built-in sort $O(n \log n)$, কিন্তু আপনি চাইলে আমি নিজে লিখে দেখাতে পারি।"

প্রশ্ন 9: Counting sort কখন ব্যবহার করবে, কখন করবে না?

যখন integer আর মানের রেঞ্জ (k) ছোট — $O(n + k)$ সময়ে sort হয়, তুলনা ছাড়াই। k অনেক বড় হলে (যেমন 10^9) বিশাল মেমোরি লাগে, তখন ব্যবহার করবো না। দশমিক বা string-এ সরাসরি চলে না।

প্রশ্ন 10: In-place sorting কী?

যে sorting ইনপুট array-র ভেতরেই কাজ করে, আলাদা বড় array লাগে না ($O(1)$ বা $O(\log n)$ extra space)। Bubble, Selection, Quick in-place; Merge sort না ($O(n)$ লাগে)।

প্র্যাকটিস প্রবলেম (নিজে সলভ করো)

আগে নিজে ২০ মিনিট চেষ্টা করো। আটকে গেলে হিন্ট দেখো। তারপরও না পারলে সলিউশন দেখে, পরদিন আবার নিজে লেখো।

1. First and Last Position of Element (Medium · LeetCode 34)

Sorted array-তে target-এর প্রথম আর শেষ index বের করো। $[5, 7, 7, 8, 8, 10]$, $8 \rightarrow [3, 4]$

হিন্ট: দুইবার binary search। প্রথমটায় পেলে $high = mid - 1$ করে বামে খুঁজতে থাকো; দ্বিতীয়টায় $low = mid + 1$ করে ডানে।

2. Sqrt(x) (Easy · LeetCode 69)

$\text{sqrt}()$ ছাড়া x -এর বর্গমূলের integer অংশ বের করো। $8 \rightarrow 2$

হিন্ট: 1 থেকে x পর্যন্ত binary search: $mid \times mid \leq x$ হলে উত্তর mid হতে পারে, ডানে যাও।

3. Sort Colors (Medium · LeetCode 75)

শুধু 0, 1, 2 আছে এমন array এক পাসে sort করো। $[2, 0, 2, 1, 1, 0] \rightarrow [0, 0, 1, 1, 2, 2]$

হিন্ট: সহজ উপায়: counting sort। Optimized: তিনটা pointer (low, mid, high) — Dutch National Flag।

4. Merge Sorted Array (Easy · LeetCode 88)

দুইটা sorted array এক করে sorted রাখো।

হিন্ট: Merge sort-এর $\text{merge}()$ ফাংশনটাই! বা পেছন থেকে বড়টা বসাও।

5. Find Peak Element (Medium · LeetCode 162)

এমন একটা index খোঁজো যার মান দুই পাশের চেয়ে বড়, $O(\log n)$ -এ।

হিন্ট: $\text{nums}[mid] < \text{nums}[mid+1]$ হলে ডানে peak আছে ($low = mid+1$), নইলে বামে ($high = mid$)।

6. Insertion Sort নিজে লেখো (Easy)

তাস সাজানোর মতো: প্রতিটা element-কে তার আগের sorted অংশে ঠিক জায়গায় ঢোকাও।

হিন্ট: $\text{key} = \text{arr}[i]$; $j = i-1$ থেকে বামে যাও, key-এর চেয়ে বড়দের এক ঘর ডানে সরে, তারপর key বসাও।

চিট শিট — দিন ২

Sorting তুলনা টেবিল (মুখস্থ রাখো)

অ্যালগরিদম	Best	Average	Worst	Space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	হ্যাঁ
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	না
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	হ্যাঁ
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	হ্যাঁ
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	না
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	হ্যাঁ*

Binary Search টেমপ্লেট

PHP — টেমপ্লেট

```
$low = 0; $high = count($nums) - 1;
while ($low <= $high) {
    $mid = $low + intval($high - $low, 2);
    if ($nums[$mid] === $target) return $mid;
    elseif ($nums[$mid] < $target) $low = $mid + 1;
    else $high = $mid - 1;
}
return $low; // না পেলো: insert position (বা -1 রিটার্ন করো)
```

এক লাইনে মনে রাখো

- **Bubble:** পাশাপাশি swap, বড়টা শেষে ভেসে যায়
- **Selection:** সবচেয়ে ছোটটা বেছে সামনে
- **Merge:** অর্ধেক → sort → merge (two pointer)
- **Quick:** pivot → ছোট বামে, বড় ডানে → recursion
- **Counting:** প্রতিটা মান কতবার — গুনে লেখো

আজকের নিজেকে যাচাই

- ☐ Binary search ২ মিনিটে bug ছাড়া লিখতে পারি
- ☐ Bubble sort (optimized) লিখতে পারি
- ☐ Merge sort লিখতে ও $n \log n$ ব্যাখ্যা করতে পারি
- ☐ Quick sort-এর worst case ব্যাখ্যা করতে পারি
- ☐ Sorting তুলনা টেবিল মনে আছে